

File	Comments
cfg_app.cpp	The default servo angles should be tied to those specified in config.gncc.allocc.ser_default_ang_deg; motors can be hard-coded to 0
GNC/bus.hpp	The CAL_FEEDBACK bus needs to go, see comments on NAV.cpp
NAV.cpp	- Pull IMU calibration out of nav; instead, the code should reside in the configurator. We grab n IMU frames (gyro, accel), process them inside of a python helper tied to the configurator to generate biases that are saved in the NAVc config object in a sub-struct called IMU_CALc. Then the params are saved and a restart is issued - In this python helper here is what we'll do 1) gyro: average each axis over n samples; that is the bias 2) accel: a) average over n samples b) from average, derive a unit vector c) multiply that unit vector by g to a high degree of precision d) subtract out that vector from the averages; that is the bias - In the compensator, we need to rotate then apply bias since we'll determine bias via the rotated IMU signals. Also, if the biases are nonzero to begin with, we will need to zero them pre-calibration. Since this may require a restart, the configurator should indicate a grayed-out calibrate button if the biases are nonzero, and include a "zero biases" button to quickly zero them. Ensure a tool tip exists on the calibrate button if you hover over it indicating that biases must be zeroed pre-calibration
ALLOC.cpp	Add a max_motor_frac to the config and enforce it here
VSM.cpp	Switch statement for the flight mode since it's not a bool
DEBUG.cpp	Remove entirely
IMU.cpp	Remove bias application; this should happen in NAV, not HAL
RCRX.cpp	Line 40; hard-coded culled signal telemetering. Rather than hard-coding this please put it in RCRXc and mirror the decimation logic in CTL.cpp
RPI.cpp	For now, remove everything in here, or make it do basically nothing.
SER.cpp	Remove the pin != 255 crap, we don't have it for the motors, we shouldn't have it here
HAL/cfg	- Make the RPI config a dummy struct with basically nothing in it for now; leave it, but don't populate it with anything - Remove debug config
HAL.cpp	- Break the update method into 2 sections 1) package ALLb_kml 2) populate HALb_k
Configurator (all files)	- Right now, we're parsing ascii over serial that was telemetered from debug.cpp. We are not doing that anymore. - Now, we'll use Google's FlatBuffers to parse the packets getting sent to the computer's serial port. Utilities to do so should exist in there, as I read

	that it's cross platform; consult me if you have issues finding the resources you need.
--	---

New code

PKG module	• Lives in HAL, in its own folder • Function • Takes in a struct and a serial port; compiles the struct into a binary packet, and sends it over the serial port • Agnostic to the struct it receives • Uses Google's FlatBuffers algorithm; library has been downloaded/included in the lib folder
------------	--